

bitcoin Coders

Tarefa Prática Mentoria Aula 02

Objetivo

Na Aula 2, evoluímos o sistema de:

- um painel baseado em RPC (estado) para
- um sistema que também observa eventos em tempo real (ZMQ)

Agora, o objetivo é dar o próximo passo:

parar de apenas receber eventos e estruturar o fluxo do sistema, além de executar sua aplicação em um ambiente acessível externamente.

Você irá evoluir seu backend para construir uma **camada de processamento orientada a eventos**, conectando:

```
evento → interpretação → estado derivado
```

E também deverá:

executar seu sistema fora do ambiente local (VPS ou túnel), tornando-o acessível via HTTP.

Contexto

Até aqui, você já sabe que:

- RPC fornece uma **fotografia do estado atual**

- ZMQ fornece um **fluxo de acontecimentos**
- eventos:
 - não são garantias
 - não são ordenados
 - não são confirmação

Além disso, sistemas reais:

- não rodam localmente
- não dependem do seu computador
- são acessados via rede

Esta atividade representa exatamente isso:

transformar eventos em informação estruturada **e operar esse sistema em um ambiente real.**

O que você deve desenvolver

Você deverá evoluir o sistema da Aula 2 implementando os seguintes componentes:

1 Estrutura de eventos no backend

Crie uma estrutura interna que registre eventos recebidos via ZMQ.

Você deve manter em memória:

- últimos blocos observados
- últimas transações observadas
- timestamp de cada evento

 Pode utilizar:

- listas
- filas (deque)
- buffers limitados

2 Endpoint `/api/events/summary`

Crie um endpoint que resume a atividade recente observada via ZMQ.

 Deve retornar pelo menos:

- número de eventos de bloco (últimos N segundos ou últimos N eventos)
- número de eventos de transação
- timestamp do último evento recebido
- taxa de eventos (ex.: tx por segundo)

Exemplo esperado:

```
{
  "blocks_observed":3,
  "tx_observed":120,
  "last_event_time":1712345678,
  "tx_per_second":4.2
}
```

3 Endpoint `/api/events/latest`

Crie um endpoint que retorna os eventos mais recentes.

 Deve incluir:

- últimos blocos (hash + timestamp)
- últimas transações (txid + timestamp)

Exemplo:

```
{
  "blocks": [
    {"hash":"abc...", "ts":1712345600},
    {"hash":"def...", "ts":1712345678}
  ],
  "txs": [
    {"txid":"tx1...", "ts":1712345670},
    {"txid":"tx2...", "ts":1712345675},
    {"txid":"tx3...", "ts":1712345678}
  ]
}
```

4 Endpoint `/api/events/state-comparison`

Comparação entre fluxo e estado

Evolua o sistema para deixar explícito:

diferença entre o que foi observado (ZMQ) e o estado atual (RPC)

Você deve:

- comparar:
 - último bloco visto via ZMQ
 - `getbestblockhash` via RPC
- indicar se há divergência

Exemplo:

```
{
  "best_block": "...",
  "last_seen_block": "...",
  "divergence": true
}
```

5 Atualização do frontend

Adicione ao seu painel:

Card: Event Activity

- número de tx recebidas
- número de blocos recebidos
- taxa de eventos

Card: Últimos Eventos

- lista das últimas transações
- lista dos últimos blocos

Indicador de divergência

- destaque visual quando:

- `bestblockhash != last_seen_block`

6 Executar o sistema em ambiente externo

Você deve disponibilizar sua aplicação para acesso externo.

Escolha uma das opções:

A VPS (recomendado)

Utilize um provedor como:

- Hetzner
- AWS
- Oracle Cloud
- DigitalOcean

Passos esperados:

- criar a VPS
- acessar via SSH
- instalar o Bitcoin Core
- configurar RPC e ZMQ
- subir o backend
- garantir que a aplicação esteja acessível

B Alternativa: Cloudflare Tunnel

Caso não utilize VPS, você pode expor sua aplicação local. Algo como:

```
cloudflared tunnel --url http://localhost:8000
```

✓ Resultado esperado

Sua aplicação deve estar acessível externamente, por exemplo:

```
http://IP_DA_VPS:8000
```

ou via URL pública do tunnel.

Critério de conclusão

- aplicação acessível via internet
- endpoints funcionando (`/api/...`)
- frontend consumindo a API externa